

Guía de Actividades Práctico Experimentales Nro. 002

1. Datos Generales

Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Arquitectura de Sistemas Distribuidos (comunicación cliente-servidor con archivos)
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Lunes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender la arquitectura de sistemas distribuidos mediante el desarrollo independiente de sus componentes (cliente y servidor).

3. Materiales y reactivos:

Guia practica

Se requiere un sistema que permita crear un servidor y un cliente con un canal de comunicación (archivo de texto), se debe evidenciar que se esta llenado el archivo.

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación

FEIRNNR - Carrera de Computación

- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

Parte 1: Desarrollo del SERVIDOR (1 hora 20 minutos)

Este módulo se desarrolla de forma independiente

Actividad

1. Crear un archivo:

servidor.py

2. Crear un archivo auxiliar:

entrada_servidor.txt

3. Implementar un programa que:

- Revise periódicamente el archivo entrada_servidor.txt.
- Detecte si existe contenido nuevo.
- Procese el mensaje:
 - Ejemplo: convertir texto a mayúsculas o invertirlo.
- Escriba el resultado en:

salida_servidor.txt

4. Condiciones:

- Debe ejecutarse continuamente (simulación de servicio activo).
- No debe reprocesar el mismo mensaje.
- Debe manejar archivos vacíos.

Parte 2: Desarrollo del CLIENTE (1 hora)

Se desarrolla sin necesidad de ver el código del servidor

Actividad

1. Crear archivo:

cliente.py

2. Implementar un programa que:

- Solicite un mensaje al usuario.
- Escriba el mensaje en:
entrada_servidor.txt
- Espere un tiempo (simulando latencia).
- Lea la respuesta desde:

salida_servidor.txt

- Muestre el resultado en pantalla.
3. Condiciones:
- No debe asumir cómo funciona el servidor internamente.
 - Solo debe conocer los archivos de entrada y salida.

Parte 3: Integración del sistema (25 minutos)

1. Ejecutar primero el servidor.

FEIRNNR - Carrera de Computación

2. Ejecutar el cliente.
3. Enviar varios mensajes.
4. Verificar:
 - Flujo correcto
 - Respuesta adecuada

NOTA: los archivos de comunicación deben estar en una sola carpeta fuera de ambos proyectos

6. Resultados esperados:

Al finalizar la práctica, el estudiante entregará:

- Código fuente de servidor.py y cliente.py (repositorio Git).

Repositorio: <https://github.com/Cristhval/SErvidores>

- Capturas de pantalla que evidencien:

(a) el archivo entrada_servidor.txt siendo llenado por el cliente,

```

Explorer
  SErvidores
    cliente
      cliente.py
    comunicacion
      entrada_servidor.txt
      salida_servidor.txt
    servidor
      servidor.py

cliente.py
cliente > cliente.py > ...
1 import os
2 import time
3
4 # Rutas a la carpeta compartida

Problems Output Debug Console Terminal Ports
PS C:\Users\HP\Saved Games\SErvidores> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.9.0.0_q-bz8x2sk43tg\python.exe C:\Users\HP\Saved Games\SErvidores\cliente\cliente.py
=====
CLIENTE DE MENSAJES
=====
Escribe 'salir' para terminar.

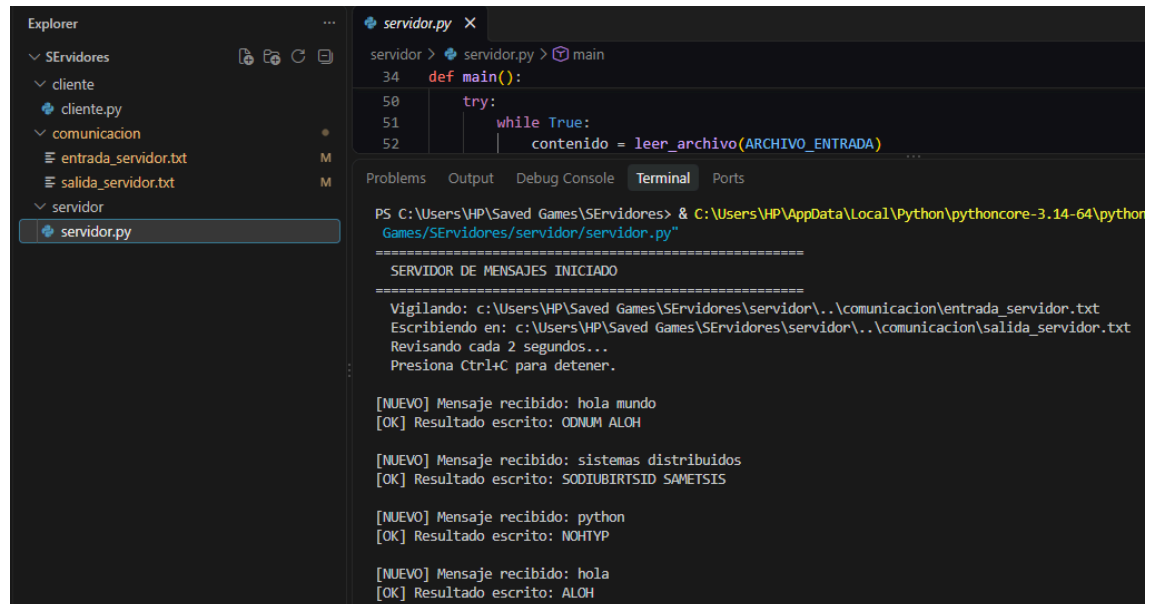
Ingresa un mensaje: hola mundo
Mensaje enviado: 'hola mundo'
Esperando respuesta (3s)...
Respuesta del servidor: ODNUM ALOH

Ingresa un mensaje: sistemas distribuidos
Mensaje enviado: 'sistemas distribuidos'
Esperando respuesta (3s)...
Respuesta del servidor: SODIUBIRTSID SAMETSIS

Ingresa un mensaje: python
Mensaje enviado: 'python'
Esperando respuesta (3s)...
Respuesta del servidor: NOHTYP

Ingresa un mensaje: hola
Mensaje enviado: 'hola'
Esperando respuesta (3s)...
Respuesta del servidor: ALOH
  
```

(b) salida_servidor.txt con el resultado procesado,



The screenshot shows a code editor with a file explorer on the left and a terminal on the right. The file explorer shows a project structure with folders 'SErvidores', 'cliente', and 'comunicacion'. The 'comunicacion' folder contains 'entrada_servidor.txt' and 'salida_servidor.txt'. The 'SErvidores' folder contains 'servidor.py'. The terminal shows the execution of 'servidor.py' with the following output:

```
PS C:\Users\HP\Saved Games\SErvidores> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python
Games\SErvidores\servidor\servidor.py"
=====
SERVIDOR DE MENSAJES INICIADO
=====
Vigilando: c:\Users\HP\Saved Games\SErvidores\servidor\..\comunicacion\entrada_servidor.txt
Escribiendo en: c:\Users\HP\Saved Games\SErvidores\servidor\..\comunicacion\saldida_servidor.txt
Revisando cada 2 segundos...
Presiona Ctrl+C para detener.

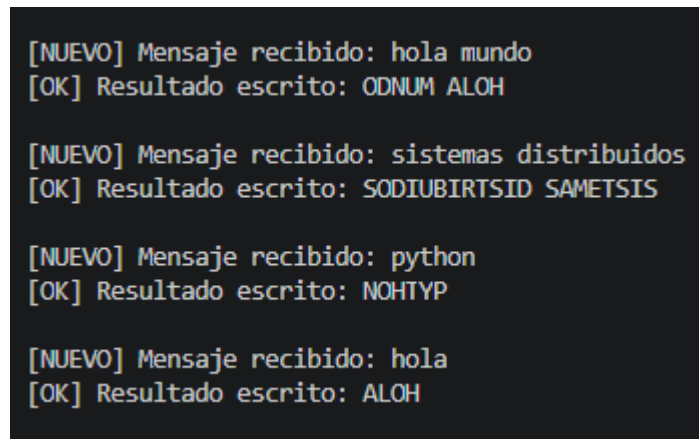
[NUEVO] Mensaje recibido: hola mundo
[OK] Resultado escrito: ODNUM ALOH

[NUEVO] Mensaje recibido: sistemas distribuidos
[OK] Resultado escrito: SODIUBIRTSID SAMETSIS

[NUEVO] Mensaje recibido: python
[OK] Resultado escrito: NOHTYP

[NUEVO] Mensaje recibido: hola
[OK] Resultado escrito: ALOH
```

(c) al menos 3 mensajes procesados correctamente.



The screenshot shows a terminal window with the following output:

```
[NUEVO] Mensaje recibido: hola mundo
[OK] Resultado escrito: ODNUM ALOH

[NUEVO] Mensaje recibido: sistemas distribuidos
[OK] Resultado escrito: SODIUBIRTSID SAMETSIS

[NUEVO] Mensaje recibido: python
[OK] Resultado escrito: NOHTYP

[NUEVO] Mensaje recibido: hola
[OK] Resultado escrito: ALOH
```

- Respuestas a las preguntas de control en el reporte de práctica. Con ello demostrará: el desacoplamiento entre cliente y servidor, la comunicación indirecta por medio compartido, y las limitaciones del modelo (tiempo real, conflictos de escritura, dependencia del medio).

7. Preguntas de Control:

¿Qué significa que cliente y servidor estén desacoplados?

significa que estos no dependen del código del otro es decir el cliente solo conoce archivo de entrada/salida y no sabe cómo el servidor procesa el mensaje.

¿Qué ventajas tiene desarrollar componentes por separado?

Las ventajas son:

- Se puede trabajar en paralelo
- Facilidad de mantenimiento
- Facilidad de hacer pruebas unitarias

¿Qué problemas pueden surgir al integrar sistemas

independientes?

los problemas pueden ser:

- condiciones de carrera para la lectura y escritura de un mismo archivo.
- falta de sincronización
- Diferentes formatos de datos

¿Qué sucede si el servidor no está ejecutándose?

Si no se está ejecutando el cliente lee salida_servidor y solo obtienen un mensaje vacío o viejo pero en sistemas reales el mensaje quedaría en cola hasta que vuelva el servicio.

¿Cómo se simula la comunicación en esta práctica?

En la práctica se usa un medio compartido en este caso se usó archivos de texto dentro de una carpeta común.

¿Qué limitaciones tiene este modelo frente a uno real?

las limitaciones son:

- solo hay un sólo lector y escritor
- el servidor debe preguntar constantemente lo que es un desperdicio de recursos
- Si el archivo se corrompe o se llega a borrar puede fallar.

¿Cómo se podría mejorar la sincronización?

Se podría usar las notificaciones por eventos (watchdog) para reemplazar el polling y emplear las colas de mensajes.

¿Qué pasaría si múltiples clientes usan el sistema?

Si múltiples clientes usarán el sistema habría condición de carrera para escribir en entrada_Servidor y podría sobrescribir el uno al otro.

8. Evaluación

NO APLICA

9. Bibliografía

Semana 2. Guaman, Jose. 2026

FEIRNNR - Carrera de Computación

10. Elaboración y Aprobación

Elaborado por	José Guamán Docente	
Ejecutado Por	Rene Rivera Técnico Docente	

Aprobado por	Edison L Coronel Romero Director de Carrera	
-------------------------	---	--